

Style Checking and Testing

COMS W4156 (Fall 2024)

Shivam Chaturvedi (sc5588)

Shreya Kochar (sk5235)

Out of scope

1. How to write code in C++/Java
2. How to build code in C++/Java

What is Style Checking?



- Style checks are a form of informal rules set by software developers.
- They ensure a certain level of predictability and consistency in a codebase.
- This in turn, improves development pace and maintainability.
- Following the set rules and practices allows for easy onboarding of new developers to the codebase.

Goals of Style Checking

- **Code Consistency:** Ensures code follows a structure across a project, making it easier to read and maintain.
- **Readability:** Improves the clarity of the code, making it more understandable for other developers or future maintenance.
- **Enforces Best Practices:** Promotes the use of common coding conventions and standards, helping to avoid errors and improve efficiency.

Style Checking

By following consistent coding practices, we can allow for integration of linters and style checking tools. If the codebase follows the set practices, these tools ensure no one breaks it going forward.

Style Checking tools or Linters in a development pipeline allow us to:

1. Catch preliminary bugs
2. Auto formatting of code
 - a. This improves readability and reduces cognitive load on the code reviewer

“Code is read much more often than it is written”

Style Checking in Java

Indentation

Correct

```
public class Example {  
    public void method() {  
        if (true) {  
            // Correct indentation  
        }  
    }  
}
```

Incorrect

```
public class Example {  
    public void method() {  
        if (true) {  
            // Incorrect indentation  
        }  
    }  
}
```

Brace placement

Correct

```
public class Example {  
    public void method() { // Correct placement of braces  
        if (true) {  
            // Do something  
        }  
    }  
}
```

Brace placement

Incorrect

```
public class Example
{
    public void method()
    {
        if (true)
        {
            // Braces on the wrong line
        }
    }
}
```

Variable Names

Correct

```
int totalScore = 0; // Good variable name, camelCase
```

Incorrect

```
int t = 0; // Single-letter variable names are not meaningful  
int TOTAL_SCORE = 0; // ALL CAPS should only be for constants, not variables
```

Comments

Correct: comments provide meaningful insight

```
/**
 * Adds two integers, ensuring that no integer overflow occurs.
 * If an overflow is detected, an exception is thrown.
 *
 * @param a the first integer
 * @param b the second integer
 * @return the sum of a and b
 * @throws ArithmeticException if the sum overflows
 */
public int add(int a, int b) throws ArithmeticException {
    if (a > Integer.MAX_VALUE - b) {
        throw new ArithmeticException("Integer overflow");
    }
    return a + b;
}
```

Comments

Incorrect: comments are unnecessary as method's functionality is obvious

```
// Calculate the sum of two numbers
public int add(int a, int b) {
    // Add a and b
    return a + b; // Return the sum
}
```

Style Checking in C++

Style Checking in C++

Good reading material on common coding best practices in C++:

1. [Google C++ Style Guide](#)
2. [Mozilla C++ Coding Style](#)
3. [C++ Core Guidelines](#) - by Prof. Bjarne Stroustrup and Herb Sutter

We will be following [cpplint](#) as part of enforcing style checks for mini projects. CPPLint is a linter by Google which follows their C++ Style Guide.

You are free to use any additional Linter and Static analysis tools to assist you.

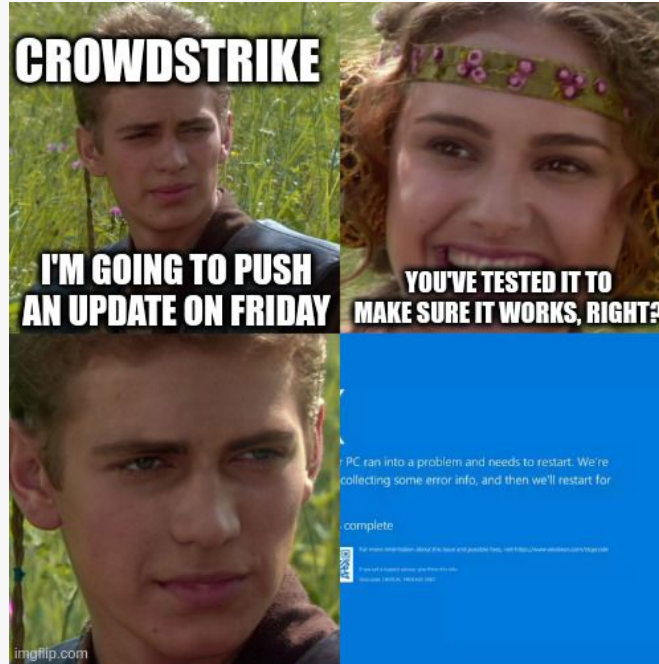
Bonus: Golang also has a very well defined coding standards, called [Effective Go](#). Feel free to read it to understand how coding styles differ between similar languages



Code Testing

1. Testing helps identify bugs, errors, and issues early in the development process, allowing developers to fix them before they become more problematic. This leads to higher quality code overall.
2. Well-tested code is typically easier to maintain and modify over time. Tests serve as documentation and help developers understand how different parts of the code should work.
3. Having a comprehensive test suite gives developers confidence to refactor and improve code, knowing they can quickly verify if changes broke existing functionality.
4. Automated tests are crucial for implementing CI/CD pipelines, allowing for faster and more reliable software releases. (We will cover this in future sessions)

Always test your code ~~or~~ and write tests for new code



Types of Tests

1. Unit Tests – Tests a single point or method. All external dependencies are mocked
2. Integration Tests – Tests how a system interacts with other systems without mocking
3. Load Tests – Test how a system handles load. You basically bombard a system on production-like environment with varying load
4. Smoke Tests – A basic integration test to check if system behaves in an expected way to invocation.

While the each definition themselves get a little bent and interchanged from one organisation to another, unit tests and integration tests are two of the most important types of tests to keep in mind.

We will cover them in details in future. This is just a primer.

Types of Tests (Contd.)

1. Acceptance Tests – A kind of integration test that tests a feature end to end. This is a use case specific test rather than a system interaction test like an integration test.
2. Regression Tests – These are tests added after fixing a bug (which is generally also followed along with an RCA) to ensure that it does not occur again. These could be unit tests or integration tests. Regression tests are also used to test features before and after a change to ensure that the behavior is unchanged.

While the each definition themselves get a little bent and interchanged from one organisation to another, unit tests and integration tests are two of the most important types of tests to keep in mind. Also, do not consider this list as an exhaustive list.

We will cover them in details in future. This is just a primer.

Demo



Car class

Methods needed in car class:

- startEngine()
 - stopEngine()
 - refuel(double amount)
 - drive(double distance)
 - isEmpty()
 - getRemainingRange()
 - needsMaintenance()
- 

Car class

Attributes needed in car class:

- **make** (`String`): The manufacturer of the car (e.g., Toyota, Ford).
- **model** (`String`): The specific model of the car (e.g., Corolla, Mustang).
- **year** (`int`): The manufacturing year of the car.
- **fuelCapacity** (`double`): The maximum amount of fuel (in gallons or liters) the car's tank can hold.
- **fuelLevel** (`double`): The current amount of fuel in the tank.
- **mileage** (`double`): The total distance the car has driven (in miles or kilometers).
- **fuelEfficiency** (`double`): The fuel efficiency of the car, typically measured in miles per gallon (MPG) or kilometers per liter (KPL).
- **isEngineOn** (`boolean`): A flag to check whether the engine is running or not.

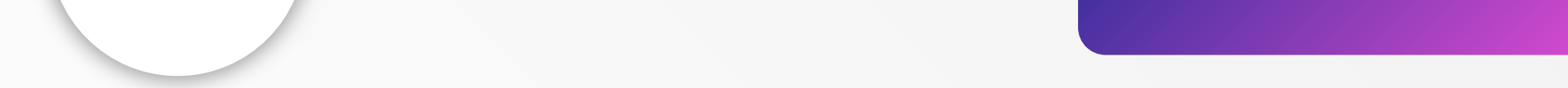


Necessary unit tests

- **Test Initial Fuel Level:** Ensures the initial fuel level is 0.
 - **Test Refuel:** Tests refueling the car and checks if the fuel level is updated correctly.
 - **Test Refuel Beyond Capacity:** Ensures that refueling beyond the fuel tank capacity is handled correctly.
 - **Test Drive:** Ensures that driving consumes fuel based on a fixed fuel efficiency (20 miles per gallon).
 - **Test Drive Without Enough Fuel:** Ensures that the car cannot drive if there's not enough fuel.
 - **Test isEmpty:** Checks if the car correctly reports whether the fuel tank is empty.
 - **Test Negative Refuel:** Ensures that attempting to refuel a negative amount throws an exception.
- 

Code Reference:

<https://github.com/DestroyerAlpha/COMS4156-demo>



Questions?

**Reach out to the team
on Edstem or Discord**

